

AOS Assignment 3: Peer-to-Peer Distributed File Sharing System

Monsoon 2025

Shubahdeep Mandal
2025201056

October 2, 2025

Contents

1	Architectural Overview	2
2	Implementation Approach & Design Choices	2
2.1	Tracker System	2
2.1.1	Synchronization: Primary-Backup Model	2
2.2	Client System	3
2.2.1	State Management & Portability	3
2.2.2	Direct Path Seeding	3
3	Detailed Implementation	3
3.1	Data Structures	3
3.1.1	Tracker Data Structures	3
3.1.2	Client Data Structures	4
3.2	Threading and Synchronization	4
3.2.1	Tracker Threading & Locking	4
3.2.2	Client Threading Infrastructure	4
3.3	Robustness Guarantees	5
4	Key Algorithms	5
4.1	Piece Selection Strategy	5
4.1.1	Pluggable Architecture	5
4.1.2	Runtime Strategy Selection	6
4.1.3	Rarest-First Algorithm Implementation	6
5	Synergistic Swarm Efficiency	6
5.1	The Leecher as a Seeder: Partial Seeding	7
5.2	The Synergy with Rarest-First	7
6	Performance Analysis	8
6.1	Download Time vs. File Size (Varying Peers)	8
6.2	Scalability Analysis (1GB File)	8
6.3	Observations and Trends	8
7	Challenges and Solutions	9
8	Compilation and Execution	10
8.1	Compilation	10
8.2	Execution	10
A	Network Protocol Design	11
A.1	Client-Tracker Protocol	11
A.2	Client-Peer Protocol	11

1 Architectural Overview

This project implements a peer-to-peer (P2P) distributed file sharing system as per the assignment specification. The system operates on a hybrid P2P model, utilizing centralized trackers for metadata management and coordination while facilitating direct client-to-client connections for the actual file data transfers. This design leverages the scalability and resource distribution of P2P networks while maintaining the organizational benefits of a client-server architecture for tasks like user authentication and peer discovery.

The system is composed of two primary components:

- **Tracker System:** A fault-tolerant cluster of two tracker servers responsible for managing user accounts, group memberships, and file metadata. They maintain a synchronized state to ensure high availability and data consistency, allowing the system to function as long as at least one tracker is online.
- **Client System:** A multi-threaded application that allows users to interact with the network. Each client acts as both a downloader (a "leecher") and an uploader (a "seeder"). It handles user commands, manages local file state, and communicates directly with other peers to download or upload file pieces in parallel.

The core design prioritizes robustness, portability, and efficient resource utilization, adhering to the constraint of using only standard C++ libraries and system calls.

2 Implementation Approach & Design Choices

2.1 Tracker System

2.1.1 Synchronization: Primary-Backup Model

To achieve fault tolerance, a Primary-Backup synchronization model was implemented.

1. **Role Election:** Upon startup, each tracker attempts to connect to the other on a dedicated replication port. The first one to start fails to connect and assumes the **Primary** role. Any tracker that successfully connects to an existing Primary assumes the **Backup** role.
2. **State Replication:** The Primary tracker is responsible for all client interactions. Whenever it processes a state-changing command (e.g., `create_user`), it forwards the command string to the connected Backup tracker, which applies the same change to its own state. This is handled by the `ReplicationManager` class.
3. **Failover Safety:** The Backup tracker constantly monitors its connection to the Primary. If the connection is lost, it promotes itself to Primary. Crucially, upon promotion, it immediately calls `state.clear_sessions()` to invalidate all active user logins. This is a critical consistency measure that forces all clients to re-authenticate, ensuring the new Primary has the correct, up-to-date peer addresses for all active users and preventing any split-brain issues.

2.2 Client System

2.2.1 State Management & Portability

The client's state is designed to be persistent and portable.

- **Dynamic Home Directory:** The client is environment-aware. It defaults to a standard hidden directory in the user's home (`$HOME/.p2p_client/`), where it stores all meta-data.
- **.meta Files:** When a file is shared, its metadata, including the absolute path to the original file, is saved to a `.meta` file. On login, these files are read to repopulate the client's seeding state. The client also performs a self-healing check on startup, using `stat()` to verify that the file paths stored in the metadata are still valid and deleting any stale `.meta` files.

2.2.2 Direct Path Seeding

For simplicity and to adhere to core requirements, the system uses a direct path seeding mechanism.

- **Implementation:** When a user shares a file, the client resolves its absolute path using `realpath()` and stores this canonical path in the persistent `.meta` file. All seeding operations (reading pieces for other peers) are performed directly from this user-provided file path.
- **Trade-off:** This approach is simple and effective, but it creates a dependency on the original file's location. For the client to continue seeding, the user must not move, rename, or delete the file after it has been shared. This was a deliberate design choice to prioritize the complex networking and concurrency aspects of the assignment over more advanced file management features.

3 Detailed Implementation

3.1 Data Structures

The choice of data structures was critical for performance and correctness.

3.1.1 Tracker Data Structures

The `TrackerState` class uses `std::unordered_map` for its primary data containers due to their efficient average-case $O(1)$ time complexity for insertions, deletions, and lookups.

```
1 // Maps user_id to user information (password, login status)
2 std::unordered_map<std::string, UserInfo> users;
3
4 // Maps group_id to group information
5 std::unordered_map<std::string, GroupInfo> groups;
6
7 // Inside GroupInfo, maps logical file_name to its metadata
8 std::unordered_map<std::string, FileMetadata> files;
```

The `FileMetadata` struct contains two `std::set` containers to store the user IDs of seeders and leechers. A `std::set` was chosen for its deterministic ordering. In the event of a group owner leaving, the next owner is chosen via `*group.members.begin()`. Using a `std::set` guarantees that both the Primary and Backup trackers will elect the exact same new owner, ensuring their states remain perfectly synchronized.

3.1.2 Client Data Structures

The client relies on two key structs for managing its state:

- **SeededFileInfo:** A simple struct holding the absolute path to the original file and its `file_size`, stored in the global `seeded_files` map.
- **DownloadState:** A comprehensive struct that acts as a central control block for a single download. It contains all metadata about the file, a vector of `PeerState` objects for each connected peer, and atomic flags and progress counters. Crucially, it also holds the mutexes and condition variable required for coordinating the download threads.

3.2 Threading and Synchronization

3.2.1 Tracker Threading & Locking

The tracker employs a simple and robust synchronization model. A single `std::mutex` within the `TrackerState` object protects all shared data. Each incoming client connection is handled by a new, detached thread, and any operation that reads or modifies the state must first acquire this global lock. This coarse-grained approach was chosen for its simplicity and guarantee of correctness, preventing complex deadlocks.

The synchronization mechanism relies on two modes of communication:

- **Full State Sync:** When a Backup tracker first connects to a Primary, the Primary serializes its entire state via `serialize_state()` into a custom, multi-line string format. This entire state dump is sent as a single `FULL_STATE_SYNC` message.
- **Command Forwarding:** After the initial sync, the Primary uses incremental updates. For every state-changing command received from a client, the Primary forwards the exact command string to the Backup. To handle temporary disconnects, a `std::deque` is used as a command queue.

3.2.2 Client Threading Infrastructure

The client features a multi-layered threading model based on a Producer-Consumer pattern to handle its diverse responsibilities concurrently.

- **Main/UI Thread:** The primary thread responsible for the raw-mode terminal, handling user input, and sending commands to the tracker.
- **Peer Server Thread:** A single, detached thread that runs the `run_peer_server_loop` to accept incoming connections.
- **Download Manager (Producer):** The `start_download` function is launched in a new thread for each download. This thread acts as a **producer**, orchestrating the piece selection strategy and assigning work to worker threads.

- **Peer Workers (Consumers):** The Download Manager spawns one `peer_worker` thread for each peer. These threads act as **consumers**, waiting on a `std::condition_variable` until assigned a piece, which they then download.

3.3 Robustness Guarantees

Several solutions were implemented to ensure the system is robust against common failures and usability issues.

- **Network Reliability:** All TCP communication uses a 4-byte length-prefixed protocol, guaranteeing that complete messages are always reassembled, regardless of underlying network packet fragmentation.
- **Process Integrity:** The tracker enforces a single instance per ID by acquiring an exclusive kernel-level advisory lock using `lockf()`. This prevents port conflicts and ensures the replication logic remains stable.
- **Concurrency Safety:** The client's download manager uses a synchronization barrier (`peers_ready` atomic counter) and performs atomic disk writes using `pwrite()` to prevent file corruption.
- **UI Integrity:** A thread-safe logging function (`log_message`) with a dedicated mutex prevents asynchronous log messages from background threads from corrupting the interactive command line.
- **Portability:** By implementing a dynamic application home directory and dynamic peer port allocation, the client binary is fully portable and multiple instances can run on the same machine without conflict.

4 Key Algorithms

4.1 Piece Selection Strategy

The client's download efficiency is heavily influenced by its piece selection strategy. A sophisticated, pluggable architecture was designed to allow for both high performance and flexibility. This was achieved by implementing the **Strategy design pattern**.

4.1.1 Pluggable Architecture

An abstract base class, `PieceSelector`, was created with a single virtual method, `select_piece`.

```
1 class PieceSelector {
2 public:
3     virtual ~PieceSelector() = default;
4     virtual int select_piece(DownloadState& state) = 0;
5 };
```

This interface decouples the high-level download manager from the low-level details of any specific piece selection algorithm. Concrete strategies, such as `RarestFirstSelector`, inherit from this class and provide their own implementation of the `select_piece` method. A `std::unique_ptr<PieceSelector>` is used to hold the active strategy in the `DownloadState` struct.

4.1.2 Runtime Strategy Selection

A key feature of this implementation is the ability for the user to select a piece selection strategy at runtime. The `download_file` command was extended to accept an optional final argument:

```
download_file <group_id> <file_name> <dest_path> [rarest|sequential|random]
```

When a download is initiated, the `start_download` function parses this optional argument. It then uses a factory-like approach to instantiate the appropriate concrete `PieceSelector` class and assign it to the download's `DownloadState`. If the argument is omitted, the system defaults to the high-performance `rarest` strategy.

4.1.3 Rarest-First Algorithm Implementation

The default and most effective strategy is Rarest-First.

- **Data Collection:** A vector, `piece_rarity`, is maintained within the `DownloadState`. When each peer worker thread connects and receives a `BITFIELD` message, it iterates through the bitfield string. For every piece a peer has ('1'), it atomically increments the counter at the corresponding index in the `piece_rarity` vector.
- **Selection Logic:** The `RarestFirstSelector::select_piece` method works as follows:
 1. It iterates through all piece indices from 0 to N.
 2. It ignores any piece the client already has (`pieces_we_have`) or that is already being downloaded (`pieces_in_progress`).
 3. For the remaining candidate pieces, it finds the one with the **lowest count** in the `piece_rarity` vector.
 4. Before returning this "best piece," it performs a final check to ensure at least one of the currently connected and active peers actually has this piece. This prevents a race condition where the client might get stuck trying to download a rare piece from a peer that has just disconnected.
- **Justification:** This strategy is highly effective in P2P networks because it prioritizes the propagation of rare pieces, increasing swarm health and download resilience. It also diversifies the downloader's own collection of pieces, making them a more valuable seeder to others more quickly.

5 Synergistic Swarm Efficiency

A key design goal of this system was not just to download files, but to do so in a way that enhances the health and speed of the entire peer-to-peer network. This was achieved through the powerful synergy of two core features: the ability for a client to act as a **partial seeder**, and the implementation of the **Rarest-First piece selection algorithm**. This combination ensures a client becomes a valuable contributor to the swarm from the very first piece it downloads.

5.1 The Leecher as a Seeder: Partial Seeding

In a naive system, a client only uploads (seeds) after it has finished downloading (leeching). Our implementation removes this distinction, allowing a client to upload pieces it possesses even while it is actively downloading others. This dual-role capability is handled by the seeder logic in the `handle_peer_connection` function.

When a request is received for a file that is still being downloaded (i.e., it exists in the `g_downloads` map), the client constructs a `BITFIELD` response that accurately reflects its partial holdings by reading the `download_state->pieces_we_have` vector. This advertises the client as a partial seeder.

```
1 // When a file is an in-progress download, we act as a partial seeder.
2 std::lock_guard<std::mutex> state_lock(download_state->state_mutex);
3 bitfield_to_send.reserve(num_pieces);
4 for (bool have_piece : download_state->pieces_we_have) {
5     bitfield_to_send += (have_piece ? '1' : '0');
6 }
7 log_message("[Peer Thread] Sending PARTIAL bitfield for ", file_name);
```

Listing 1: "Constructing an accurate and partial bitfield in `client_peer.cpp`"

5.2 The Synergy with Rarest-First

The ability to act as a partial seeder is powerful, but its effectiveness is magnified exponentially when combined with the Rarest-First piece selection strategy, whose implementation was detailed in the previous section.

These two features create a highly efficient feedback loop:

1. The **Rarest-First algorithm** causes a new client to prioritize acquiring the pieces that are least available across the entire swarm.
2. The **partial seeding** mechanism allows the client to almost immediately begin sharing these newly acquired, high-value rare pieces with other peers.

This synergy rapidly transforms a new leecher from a simple data consumer into a critical node for distributing the most endangered pieces. This simultaneously strengthens the resilience of the entire file swarm and accelerates the download speed for all participants.

6 Performance Analysis

To evaluate the performance of the system, a series of tests were conducted on a local machine to measure download times under various conditions. All tests were performed using the high-performance 'rarest' piece selection algorithm. The primary metrics measured were download time versus file size, and the scalability of downloads as more peers were added to the swarm.

6.1 Download Time vs. File Size (Varying Peers)

This test measures the time taken to download files of different sizes, scaling from a single peer up to five peers.

File Size	1 Peer	2 Peers	3 Peers	4 Peers	5 Peers
1 MB	0.25s	0.16s	-	-	-
10 MB	2.08s	1.07s	-	-	-
100 MB	20.36s	10.34s	-	-	-
500 MB	94.05s	46.43s	32.08s	23.44s	11.77s
1 GB	193.49s	95.88s	66.38s	48.31s	22.82s
2 GB	-	-	71.54s	53.51s	45.80s

Table 1: Download time (in seconds) for varying file sizes and number of peers.

6.2 Scalability Analysis (1GB File)

This test focuses on the download time for a large 1GB file to clearly illustrate the impact of adding more peers.

Number of Peers	Download Time (s)	Improvement vs. Prev.
1	193.49s	-
2	95.88s	50.4%
3	66.38s	30.8%
4	48.31s	27.2%
5	22.82s	52.7%

Table 2: Scalability and performance improvement for a 1GB file download.

6.3 Observations and Trends

The collected data reveals several key performance characteristics of the system:

- **Excellent Scalability:** The system demonstrates a significant and consistent decrease in download time as more peers are added. As shown in Table 2, moving from 1 to 2 peers for a 1GB file cut the download time in half (a **50.4%** improvement), proving the effectiveness of the parallel download implementation.
- **Diminishing Returns:** While adding a 3rd and 4th peer continued to improve performance (by **30.8%** and **27.2%** respectively), the rate of improvement began to slow.

This is an expected behavior in P2P systems, suggesting that the downloader's own resources—specifically network bandwidth and disk I/O speed—are becoming the bottleneck, rather than the availability of data from seeders. The system is so effective at sourcing pieces that it begins to saturate the client's capacity to receive and write them.

- **Anomalous Speedup at 5 Peers:** Interestingly, the jump from 4 to 5 peers for the 1GB file resulted in a massive **52.7%** performance boost, which is an outlier compared to the diminishing returns trend. A likely explanation is that with five sources, the operating system's network scheduler had a much higher probability of keeping the client's network buffer full at all times, overcoming latency dips from any single peer and maximizing TCP throughput. It highlights the resilience and high potential of having a well-populated swarm.
- **Linear Scaling with File Size:** For any fixed number of peers, the download time scales linearly with the file size. For example, with a single peer, the 100MB file took ~20s and the 1GB file (10x larger) took ~193s (9.6x longer), which is a clear linear relationship. This indicates that the system has very low overhead and its performance is dictated purely by the amount of data being transferred.

This data validates that the multi-threaded, parallel downloading architecture is implemented correctly and effectively utilizes multiple peers to dramatically accelerate file transfers.

7 Challenges and Solutions

The development process was iterative, with several key challenges addressed across different phases of the project.

- **Challenge: Build Environment & Dependencies.** Early compilation failed due to linker errors (`cannot find -lcrypto`) and numerous compiler warnings about deprecated OpenSSL functions.
- **Solution:** The linker error was resolved by installing the required development library (`libssl-dev`). The deprecation warnings were addressed by refactoring the hashing utility to use the modern, recommended EVP (Envelope) API from OpenSSL, improving code quality and future-proofing the implementation.
- **Challenge: Multi-threaded UI Corruption.** Asynchronous log messages from background download threads would frequently corrupt the user's interactive command line input.
- **Solution:** A thread-safe logging function (`log_message`) was implemented. It uses a mutex to serialize access to `stdout` and intelligently redraws the user's current input line after printing each log message, providing a clean and usable interface.
- **Challenge: Download Stalls & Concurrency Bugs.** The most complex challenge was a recurring stall during multi-peer downloads.
- **Solution:** This was traced to a race condition where the main download thread would try to assign pieces before worker threads had completed their setup and received bitfields. This was resolved by implementing a synchronization barrier using an atomic counter.

(peers_ready) to ensure the main download process doesn't start assigning pieces before all worker threads have completed their handshake. Further deadlocks were resolved by implementing a proper two-way signaling Producer-Consumer model using a `std::condition_variable`.

8 Compilation and Execution

8.1 Compilation

The project uses a standard Makefile in the root directory. To compile both the tracker and client executables, run:

```
1 $ make
```

This will create binaries at `./tracker/tracker` and `./client/client`.

8.2 Execution

The system requires at least one tracker and one client.

```
1 # Start the primary tracker
2 $ ./tracker/tracker tracker_info.txt 1
3
4 # In another terminal, start the backup tracker
5 $ ./tracker/tracker tracker_info.txt 2
```

Listing 2: Terminal 1: Start Trackers

```
1 # Start the client, providing the tracker info file
2 $ ./client/client tracker_info.txt
```

Listing 3: Terminal 2: Start Client

The client will connect to the primary tracker and provide a `>>` prompt for commands.

A Network Protocol Design

The custom TCP protocol uses simple, space-delimited ASCII commands. All messages are length-prefixed to ensure reliable framing.

A.1 Client-Tracker Protocol

- `create_user <user_id> <pass>`
- `login <user_id> <pass> <peer_port>`
- `upload_file <group_id> <file_name> <size> <hashes>`
- **Response Format:** `SUCCESS: <message>` or `ERROR: <message>`

A.2 Client-Peer Protocol

- `HANDSHAKE <file_name>`
- **Response:** `HANDSHAKE_OK` or `HANDSHAKE_FAIL: <reason>`
- `BITFIELD <011010...>` (Note: The seeder intelligently constructs this bitfield, sending a partial map if it is also leeching the same file.)
- `REQUEST_PIECE <piece_index>`
- `PIECE <piece_index> <binary_data>`